

CS-311/CS-452: Analysis & Design of Algorithms

Introduction

1

Objectives

- It is a core course in a typical undergraduate Computer Science Curriculum.
- This course is designed
 - ❑ to provide students with an understanding of
 - principles
 - techniques
 - ❑ used in the **design** and **analysis** of computer algorithms.

2

Course Contents / Topics

- 1) Introduction to the course and course objectives
- 2) Algorithm Design & Analysis
- 3) Asymptotic Analysis
- 4) Recurrence Relations
- 5) Divide and Conquer Algorithm Design Strategy
- 6) Sorting Algorithms, Sorting in Linear Time
- 7) Analysis of Searching Algorithms
- 8) Hashing
- 9) Tree Algorithms
- 10) Graph Algorithms
- 11) Backtracking Algorithms
- 12) Greedy Algorithms
- 13) Dynamic Programming Design Strategy
- 14) String Matching Algorithms
- 15) Theory of NP-Completeness

3

Reference Books

T. Corman, E. Leiserson, L. Rivest, C. Stein *Introduction to Algorithms*, MIT press

R. E. Neapolitan, K. Naimipour, *Foundations of Algorithms*, Narosa Publishing House

M. A. Weiss, *Data Structures and Algorithm Analysis*, Pearson Education Inc.

R. Sedgewik, *Algorithms*, Addison-Wesley

D. E. Knuth, *The Art of Computer Programming*

4

Algorithms

History

- Procedures for solving geometric and arithmetic problems were formulated by ancient Greeks.

- Some two thousands years ago, the celebrated procedure for finding greatest common divisor (gcd) was discovered by **Euclid**.



- In tenth century, Persian mathematician **Muhammad-ibn-e- Musa Alkhowarizmi** stated procedures for solving algebraic equations. The procedure are described in his famous book *Kitab al Jabr wa'l muqabala* ('Rules of restoring and equating'). The word *algorithm* is coined from *Alkhowarizm*



- In the mid- twentieth century **D.E. Knuth** undertook in depth study and analysis of algorithms. His work is embodied in the comprehensive book '*Art of Computer Programming*', which serves as foundation for modern study of algorithms.



5

Algorithm

Formal Definition

An algorithm is an orderly **step-by-step procedure**, which has the characteristics:

- 1) It accepts **one or more** input value
- 2) It returns at **least one** output value
- 3) It terminates after **finite steps**

6

Overview

Algorithm Applications

7

Algorithms

Applications

Algorithms have been developed to solve an enormous variety of problems in many application domains. Some broad categories of algorithms are listed below. Some of these will be examined later in some detail.

- *Sorting Algorithms (Elementary and Advanced)*
- *Searching Algorithms (Linear and Non-linear)*
- *Strings Processing (Pattern matching, Parsing, Compression, Cryptography)*
- *Image Processing (Compression, Matching, Conversion)*
- *Mathematical Algorithms (Random number generator, matrix operations, etc)*

8

Algorithms

Classic Problems

A number of classic problems computer science have been widely studied. Several algorithmic approaches been attempted to find their satisfactory solutions. Some of typical best known examples are as follow.

- *Hamiltonian Circuit Problem*
- *Traveling Salesperson Problem*
- *Graph Coloring Problem*
- *Knapsack Problem*
- *The Sum-Set Problem*
- *The n-Queen Problem*

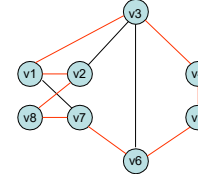
9

Classic Problems

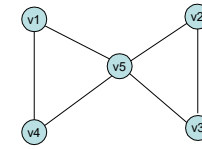
Hamiltonian Circuit Problem

Problem: Determine if a given graph has a **Hamiltonian circuit**.

- A Hamiltonian circuit, also called **tour**, is path that starts and ends at the same vertex, and passes through all other vertices exactly once.
- A graph can have more than one Hamiltonian circuit, or none at all.



Hamiltonian Circuit {v1,v2,v8,v7,v6,v5,v4,v3,v1}



No Hamiltonian Circuit

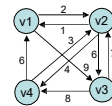
10

Classic Problems

Traveling Salesperson (TSP) Problem

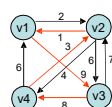
Problem(a): A salesperson has to travel to n cities, such he starts at one city and ends up at the same city, visits each city **exactly once**, and covers **minimum distance**.

Problem(b): Determine **minimum tour** for a graph with n vertices.

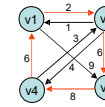


Given Graph

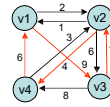
Tours starting from vertex v1



Optimum tour {v1, v3, v4, v2, v1}
Distance: $9+8+3+1=21$



Tour {v1, v2, v3, v4, v1}
Distance: $2+6+8+6=22$



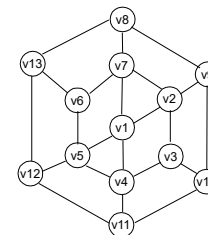
Tour {v1, v3, v2, v4, v1}
Distance: $9+7+4+6=26$

11

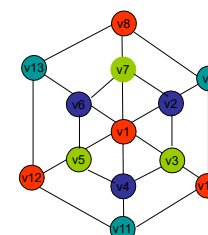
Classic Problems

Graph Coloring Problem

Problem : For given graph, find the **minimum number of colors** that can be used to color the vertices so that **no two adjacent vertices have the same color**.



Uncolored Graph



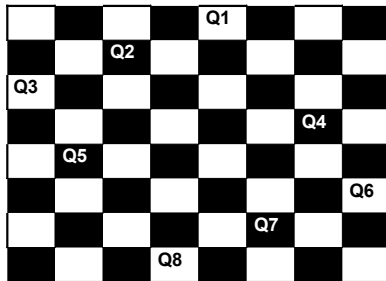
4- color graph with minimum colors

12

Classic Problems

The n-Queen Problem

Problem : n queens are to be placed on an n by n chessboard so that **no two queens threaten each other**. A queen threatens another queen if is on the **same row, or same column, or same diagonal** of the chess board.



A possible solution of 8-queen problem

13

Overview

Algorithm Design

14

Algorithm Design

Paradigm

➤ The algorithms are categorized according to the **underlying approach** to solving. The main categories are

- Divide-and-Conquer Algorithm
- Greedy Algorithm
- Backtracking Algorithm
- Dynamic Programming
- Brute Force
- Approximation Algorithm
- Randomized Algorithm

➤ Each type is generally applicable to the solution of a **range of problems**. There are, however, certain **advantages and limitations** associated with their performance.

15

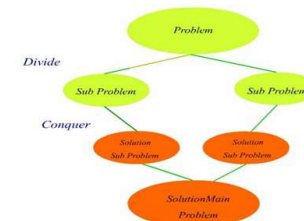
Divide-and-Conquer Algorithm

Strategy

➤ The **divide and conquer algorithm** has the following approach:

- 1) The problem is split (divided) into distinct independent subproblems
- 2) The subproblems are solved (conquered) separately
- 3) The solutions to the subproblems are merged together to obtain solution to main problem.

➤ The figure portrays the basic steps.

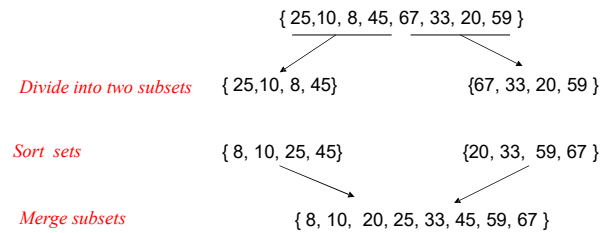


16

Divide-and-Conquer Algorithm

Problem Instance

Problem: Sort the set of numbers {25, 10, 8, 45, 67, 33, 20, 59} in ascending order.



17

Divide-and-Conquer Algorithm

Applications

➤ Divide and conquer algorithm applies to a wide range of problems. Some common applications are:

- Quick Sort
- Merge Sort
- Binary Search
- Multiplication of Large Integers
- Strassen's Matrix Multiplication

18

Divide-and-Conquer Algorithm

Limitations

➤ Divide and conquer algorithm provides efficient sorting and searching methods. It, however, has following limitations:

- The divide-and-conquer algorithm is implemented using **recursive function calls**, which is not supported by some of the programming languages.
- Recursive calls need **large stacks** to store intermediate results, utilizing computer's special memory called **heap** which may not be available on some systems.
- The divide-and-conquer algorithm is in-efficient when **subproblems are not of nearly equal size**.

19

Greedy Algorithms

Strategy

Greedy Algorithm is designed to solve **optimization problems**. Generally, the problem admits a large number of solutions, each with associated cost, which need to be **maximized** or **minimized**. The greedy algorithm has the following approach:

- 1) The problem is solved in steps. At each step, several choices are available for partial solutions. A **greedy choice** is made by selecting the option with **maximum** (or **minimum**) cost. This is referred to as **locally optimum solution**, as opposed to **globally optimum solution**, which is expected to be the overall optimum solution.
- 2) It is checked if the selected option satisfies **problems constraint** (for example, the cost does not exceed the prescribed limit). This is referred to as **feasible solution**.
- 3) If current solution is not feasible, the **next best option** is selected.
- 4) Once a choice is made, it is not changed in subsequent steps. This condition is referred to as **irrevocability**.

20

Greedy Algorithm

Problem Instance

Problem: ATM is to deliver a change for Rs. 98, with minimum number of currency notes/coins.

Choices



- 100 rupee is the maximum choice but it is no feasible because it exceeds 98
- Select 1x Rs. 50 note, balance 48
- Select 2xRs. 20 note, balance 8
- 10 rupee note is maximum choice but not feasible because it exceeds 8
- Select 1x Rs. 5 coin, balance 3
- Select 1xRs. 2 coin, balance 1
- Select 1xRe. 1 coin balance 0

Final solution : 1x Rs. 50 + 2xRs.20 + 1xRs.5 + 1xRs.2 + 1xRs.1

21

Greedy Algorithm

Applications

The greedy algorithm provides *optimum* or *suboptimum* solutions to the problems. Some typical applications are:

- Minimum Cabling in network
 - Prim's Algorithm
 - Kruskal's Algorithm
- Shortest route in a network
 - Dijkstra's Algorithm
- Huffman coding for text compression
- Optimum Job Scheduling
- Bins Packing
- 0-1 Knapsack Problem

22

Greedy Algorithm

Limitations

The greedy algorithm provides a simple and elegant solution to optimization problems. It has, however, the following limitations:

- The greedy **algorithm does not always produce a globally optimum solution**, (for example, in case of traveling salesperson problem).
- In order to check whether or not the resulting solution is optimal, **further analysis** is required.
- Often the analysis for globally optimum solution is quite complex.
- Generally, efficient implementation of greedy algorithm requires supplementary data structure, such as **priority queue**

23

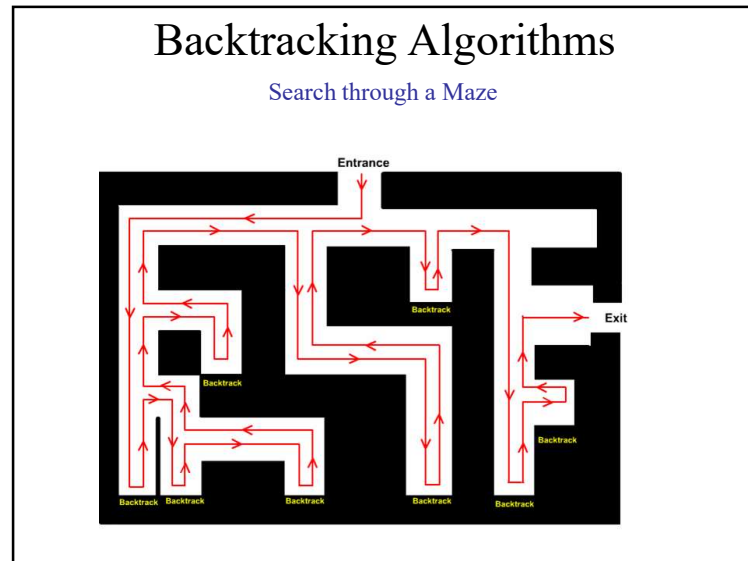
Backtracking Algorithm

Strategy

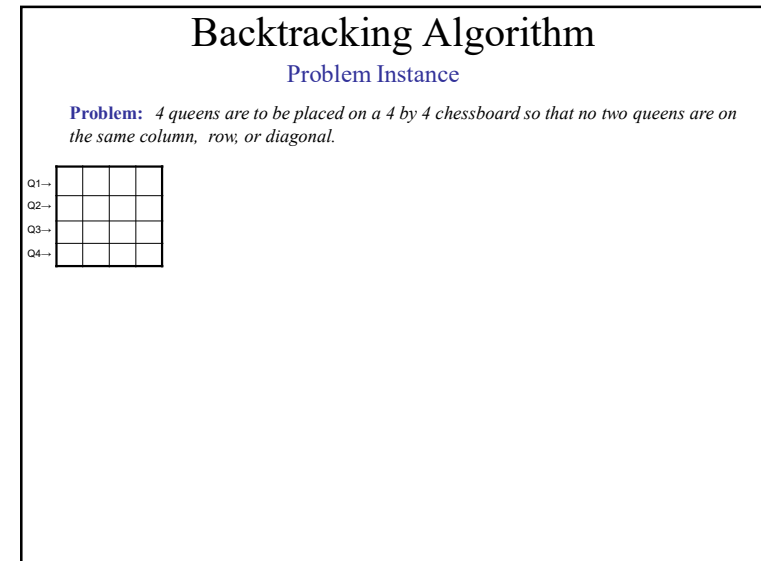
Backtracking algorithm is used to construct a feasible solution by making selections from a large set of choices. Each subset of choices corresponds to a partial solution. The algorithm has the following approach.

- 1) An initial choice is made for some **basic partial solution**.
- 2) The solution is extended by considering additional choices. The procedure is repeated until the desired solution is found, or a **dead end** is reached. In the latter case, the algorithm **backtracks to the previous position** and looks for the alternative path.
- 3) In order to backtrack, information regarding **current step** is saved on a stack.

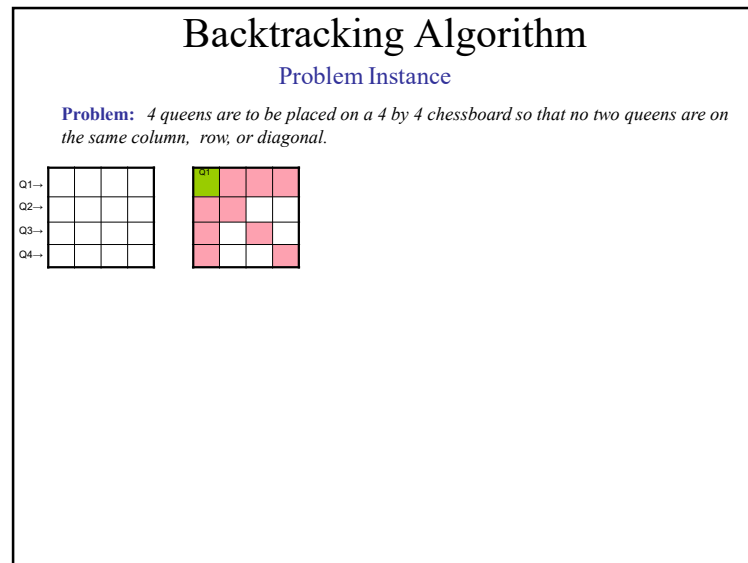
24



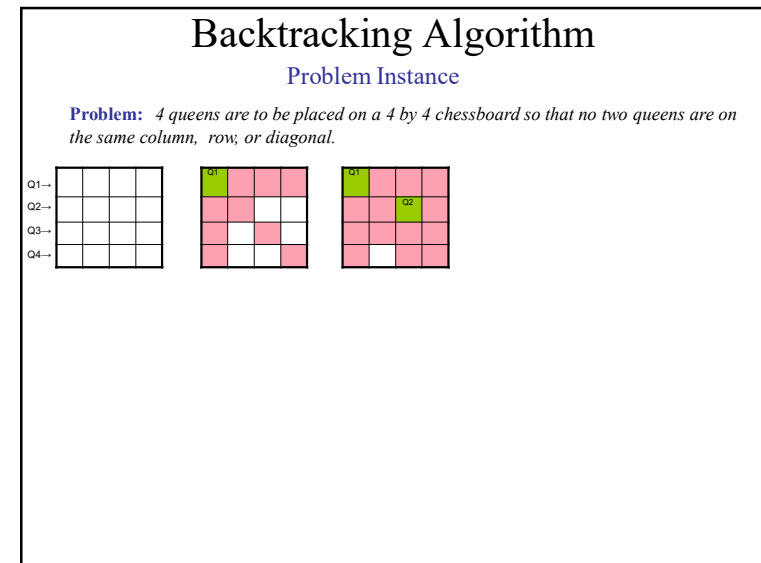
25



26



27

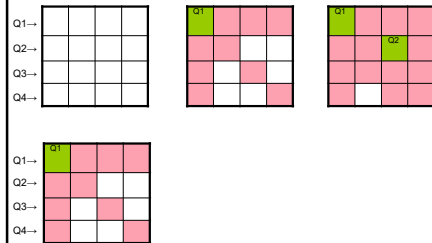


28

Backtracking Algorithm

Problem Instance

Problem: 4 queens are to be placed on a 4 by 4 chessboard so that no two queens are on the same column, row, or diagonal.



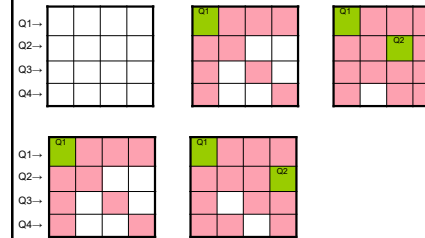
No room for Q3.
Backtrack to previous position.

29

Backtracking Algorithm

Problem Instance

Problem: 4 queens are to be placed on a 4 by 4 chessboard so that no two queens are on the same column, row, or diagonal.

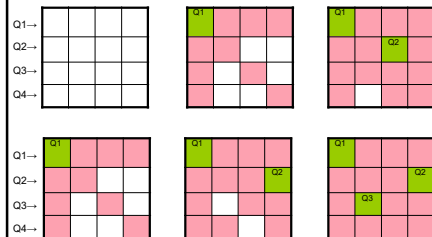


30

Backtracking Algorithm

Problem Instance

Problem: 4 queens are to be placed on a 4 by 4 chessboard so that no two queens are on the same column, row, or diagonal.



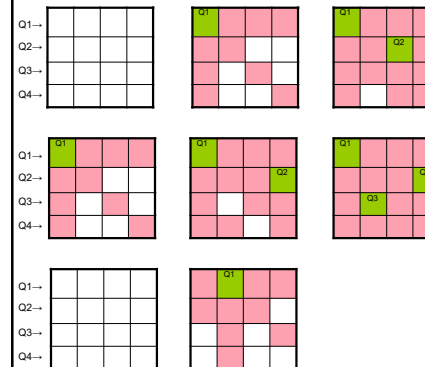
No room for Q4
Backtrack to
Original configuration

31

Backtracking Algorithm

Problem Instance

Problem: 4 queens are to be placed on a 4 by 4 chessboard so that no two queens are on the same column, row, or diagonal.



32

Backtracking Algorithms

Applications

Backtracking algorithms are useful for *solving problems in which the number of choices grows rapidly with the increase in problem size*. In some cases, the algorithm provides an *approximate solution* when exact solution is not workable. Some classic examples falling in this category are listed below.

- *n-queens problem*
- *Playing Games*
- *Sum of Subset Problem*
- *Map Coloring Problem*
- *Bin Packing Problem*
- *Traveling Salesperson problem*
- *Hamiltonian Circuit*

33

Backtracking Algorithms

Limitations

Some limitations of Backtracking algorithm are as under.

- *The algorithm generally uses a rooted tree, called **space state tree**, in which each node represents a choice and associated partial solution. A node which leads to a successful solution is called **promising**. By contrast a node that leads to dead end is called **nonpromising**. The identification of promising nodes is difficult in practice.*
- *Analysis of backtracking algorithm is difficult.*
- *Implementation of backtracking algorithm depends on tree searching methods.*

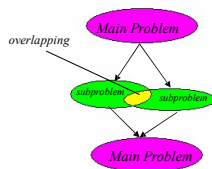
34

Dynamic Programming Algorithm

Strategy

The dynamic programming algorithm is designed to solve *optimization* problems. It follows the following approach:

- 1) *The main problem is partitioned into **overlapping subproblems**, which have some common computation. At each step the computations are **stored in a table**.*
- 2) *The problem is partitioned down to the lowest level.*
- 3) *The subproblems are **solved using values stored in the table***
- 4) *The solutions to subproblems are combined, in **bottom up** fashion, to obtain solution to the main problem*



35

Dynamic Programming Algorithm

Applications

The dynamic programming algorithm can be used to solve a wide variety of problems. Some common applications are:

- *Shortest path in a network*
- *Largest Common Sequence*
- *Matrix chain multiplication*
- *Optimum binary search tree*
- *Finding Binomial Coefficients*
- *0-1 Knapsack Problem*

36

Brute Force Algorithm

Strategy

The brute force is the simplest and straightforward method of solving a problem. It has the following approach:

- Using problem's definition and constraints, the algorithm **explores all possibilities**, to find a solution that satisfies the given conditions/requirements
- In optimization problems it examines **all feasible solutions**, one by one, and returns the one that has minimum (or maximum) associated cost
- In search problems it carries out **all possible comparisons** of given key with the problem data items.
- In sorting problems it performs comparisons **all pairs of data** items to bring them in appropriate order.

37

Brute Force Algorithm

Problem instance

Problem: Find if the word 'OR' appears in string 'BRUTE FORCE ALGORITHM' ?

Result of exhaustive search: Match found

String \ Status	B	R	U	T	E	F	O	R	C	E	A	L	G	O	R	I	T	H	M
No match found	O	R																	
No match found		O	R																
No match found			O	R															
No match found				O	R														
No match found					O	R													
No match found						O	R												
No match found							O	R											
Match found								O	R										

38

Brute Force Algorithm

Applications

Brute Force algorithms are useful to solve small instances of some common problems. Typical applications are:

- Simple Sorting
 - Selection Sort
 - Bubble Sort
 - Insertion Sort
 - Shell Sort
- String matching
- Exhaustive Search
 - Linear search
 - Sum-subset Problem
 - Hamiltonian Problem
 - Traveling Salesperson Problem.

39

Approximation Algorithms

Strategy

The approximation algorithms are designed to obtain **suboptimal** solutions to the **optimization** problems. Such algorithms are not guaranteed to produce best solution but is reasonably close to the optimal solution.

- The approximation algorithms are useful when no efficient algorithms are available to obtain optimal solution for a large inputs.
- A approximation algorithm uses **heuristics** to solve the problem. A heuristic is a **rule of thumb**, which is based on good judgment, common sense, or experience rather than on mathematical principles.
- Invariably, for optimization problems the heuristics are based on **greedy choices**

40

Approximation Algorithm

Applications

The approximation algorithms are useful to solve the problem that are *difficult*, *hard*, and *intractable*. The running time for such problems increases *exponentially* with problem size. The problems in this category are referred to as *NP-Complete*. Several NP-Complete problems have been identified. Some best known examples are:

- *Hamiltonian Circuit problem*
- *Traveling Salesperson Problem*
- *Graph Coloring Problem*
- *Knapsack Problem*
- *Bin Packing Problem*
- *Graph Clique Problem*
- *Sum-Subset Problem*

41

Approximation Algorithm

Limitations

The approximation algorithms are useful in that they can be used to reduce the exponential running times to some reasonable limit. They have, however, following the limitations:

- *The approximation algorithm provides a suboptimal solution to optimization problem. The quality of the solution is determined by **performance factor**, which is the ratio (or absolute difference) of **optimum value** and **suboptimum value**.*
- *In order to determine the quality of the solution some **additional analysis** of approximation algorithm is required.*

42

Overview

Algorithm Analysis

43

Analysis of Algorithm

Objective

The purpose of algorithm analysis is to determine:

- **Time efficiency**
Performance in terms of running times for different input sizes
- **Space utilization**
Requirement of auxiliary storage to run the algorithm
- **Correctness of algorithm**
Results are trustworthy, and algorithm is robust

44

Algorithm Analysis

Approaches

Basically three approaches can be adopted to analyze algorithm *running time* in terms of *input size*:

Empirical Approach

Running time measured experimentally

Analytical Approach

Running time estimated using mathematical modeling

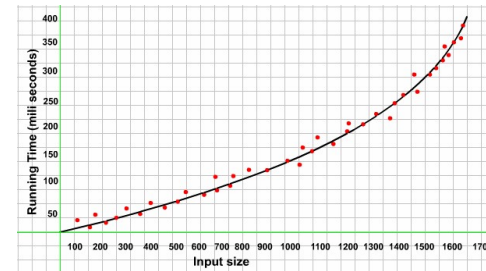
Visualization

Performance is studied through animation for different data sets

45

Empirical Analysis

The running time of algorithm is measured for different data sizes and time estimates are plotted against the input. The graph shows the trend.



46

Empirical Analysis

Limitations

Running time critically depends on:

➤ **Hardware resources used**

(CPU speed, IO throughput, RAM size etc.)

➤ **Software environment**

(Compiler, Programming Language etc.)

➤ **Program design approach**

(Conventional, Structured, Object Oriented)

47

Analytical Approach

Strategy: Running time is estimated by analyzing the *primitive operations* which make *significant contributions* to the overall algorithm time. These may broadly include:

- Assigning a value
- Comparing data items
- Computing a value
- Moving a data item
- Calling a procedure
- Returning a value

48

Algorithm Specification

Conventions

- *Plain natural language*
High level description
- *Pseudo Code*
Low level to facilitate analysis and implementation

49

Specification Using Natural Language

Preorder Tree Traversal Algorithm

- Step1.** Push tree root to stack
- Step2.** Pop the stack. If stack is empty exit, else process the node
- Step3.** Travel down the tree following the left most path, and pushing each right child onto the stack
- Step4.** When leaf node is reached, go back to step 2.

50

Pseudo Code Convention

Declaration: Algorithm procedure name with parameters e.g MERGE(A,p,q)

Assignments : Using left arrows ($\leftarrow$) e.g $j \leftarrow k \leftarrow p$

Comparisons : Using symbols $\leq \geq \neq = > <$

Logical : Using connectives **and** , **or**

Computations: Using arithmetic symbols

Exchanges: Using symbol $\leftrightarrow$ e.g $A[i] \leftrightarrow A[k]$ (swap)

Loops:

for ---- **do** ----, **for**----**downto** ----**do**---

while ---- **do**---

do ---- **until**---

Conditions: **if** ---- **then**----**else**----

Block structure: Using indentation

do---

do----

if--- **else** ----

Comments: Using symbol $\blacktriangleright$

51

Pseudo Code Convention

Example

```
1 for j  $\leftarrow$  2 to n
2   do key  $\leftarrow$  A[j]
3  $\blacktriangleright$  Insert A[j] into sorted sequence A[1..j-1]
4   i  $\leftarrow$  j - 1
5   while i > 0 and A[i] > key
6     do A[i+1]  $\leftarrow$  A[i]
7       i  $\leftarrow$  i-1
8   A[i+1]  $\leftarrow$  key
```

52

Types of Analysis

Best Case Analysis

- Determines minimum possible running time
- Uses most favorable input of given size
- Little use in practice

Worst Case Analysis

- Determines maximum possible running time
- Uses most unfavorable input of a given size
- Most desirable in practice

Average Case Analysis

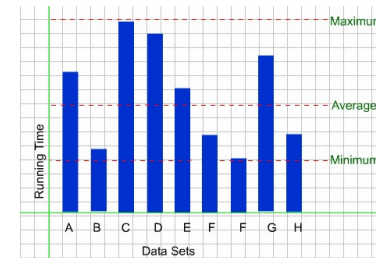
- Determines average (expected) running time
- Uses probabilistic approach
- Difficult

53

Empirical Study

Best, Average, Worst Cases Analysis

Empirical analysis can be used to estimate running times for different scenarios. The running time is measured for different data sets of the *same* size. A plot provides an indication **bounds** for maximum (best), average, and minimum (worst) running times.



54

Running Time Analysis

Linear search for value 25

Input Data Scenarios

Best case:	25	19	33	20	50	81	29	53	70	66
Average case:	70	19	81	20	29	25	50	66	33	53
Worst case:	70	20	66	33	50	81	53	19	29	25
Worst case (value non-existent)	53	70	66	20	50	81	29	33	19	16

55

Case Study

Problem

- (1) Develop an algorithm to find maximum element in an array of size n
- (2) Analyze the algorithm for:

- Time efficiency
- Space efficiency
- Correctness

56

Case Study

Algorithm

- Step #1: Store first element of the array in variable **max**
- Step #2: Scan array to compare **max** with other elements
- Step #3: Replace **max** with a larger element
- Step #4: Return value held by **max**

57

Case Study

Pseudo Code

The function **FIND-MAX** finds maximum element in an array. The array **A**, of size **n**, is passed as argument to the function.

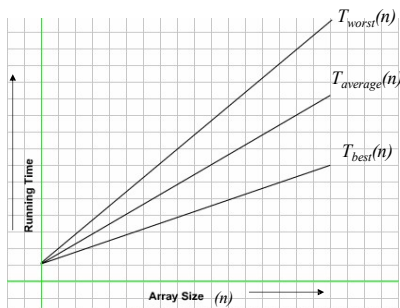
```

FIND-MAX ( A[1..n] )
1  max ← A[1]           ▶ Store first array element into variable max
2  for j ← 2 to n do     ▶ Scan remaining elements
3      if ( A[j] > max )   ▶ Compare an element with max
4          then max ← A[j] ▶ Replace max with a larger element
5  return max           ▶ Return maximum element
    
```

58

Time Analysis

Comparison of Running Times



59

Case Study

Space Efficiency

➤Space analysis of algorithm for finding maximum element is simple and straight. It amounts to determining space utilization as function of array size.

- The total space requirement consists of memory used by the program statements and array element. The former is a fixed and does not depend on array size.
- The amount of storage requirement for the array depends on the nature of data type (integer, floating point, strings). It increases in direct proportion to array size.

Thus, if $S(n)$ is storage efficiency

$$S(n) = \underbrace{A}_{\text{Program space requirement}} + \underbrace{B.n}_{\text{Array space requirement}}$$

tar

60

Algorithm Analysis

Visualization

- Algorithm *animation* and visualization techniques are used to study
 - *Inner working and operation of an algorithm*
 - *Obtain statistics about the performance*
 - *Compare performance of competing algorithms*